Hive: A Neuro-Symbolic Architecture for Portfolio Optimisation

Abhishek Bansal (2022021), Amartya Singh (2022062), Vinayak Agrawal (2022574)

Advisor - Prof. Pankaj Vajpayee

1. Abstract

Financial time-series forecasting is plagued by two fundamental issues: the **Representation Gap**, where deep learning models overfit to high-frequency noise, and the **Semantic Gap,** where numerical models fail to account for macro-economic regimes (e.g., policy shifts). This project introduces a **Neuro-Symbolic Architecture** that bridges these worlds.
We propose,
1 - **MS-DAN**, a decompositional network that generates calibrated probabilistic forecasts,
2 - **"The Hive"** a multi-agent system where Fin-R1 agents synthesize statistical outputs with a causal **Knowledge Graph** to make interpretable capital allocation decisions.

2. Problem Statement

- **Non-Stationarity**: Financial data distributions shift over time (Covariate Shift), making standard regression models (ARIMA/Linear) unreliable for long horizons.
- **The "Black Box" Risk**: End-to-end RL agents trained on raw price data lack safety guarantees and interpretability, rendering them unsuitable for institutional capital allocation.
- **The Uncertainty Problem**: Point forecasts (e.g., "Price will be $100") are mathematically insufficient for risk management. A robust system requires Quantile Estimation to calculate Value at Risk (VaR).

3. Methodology

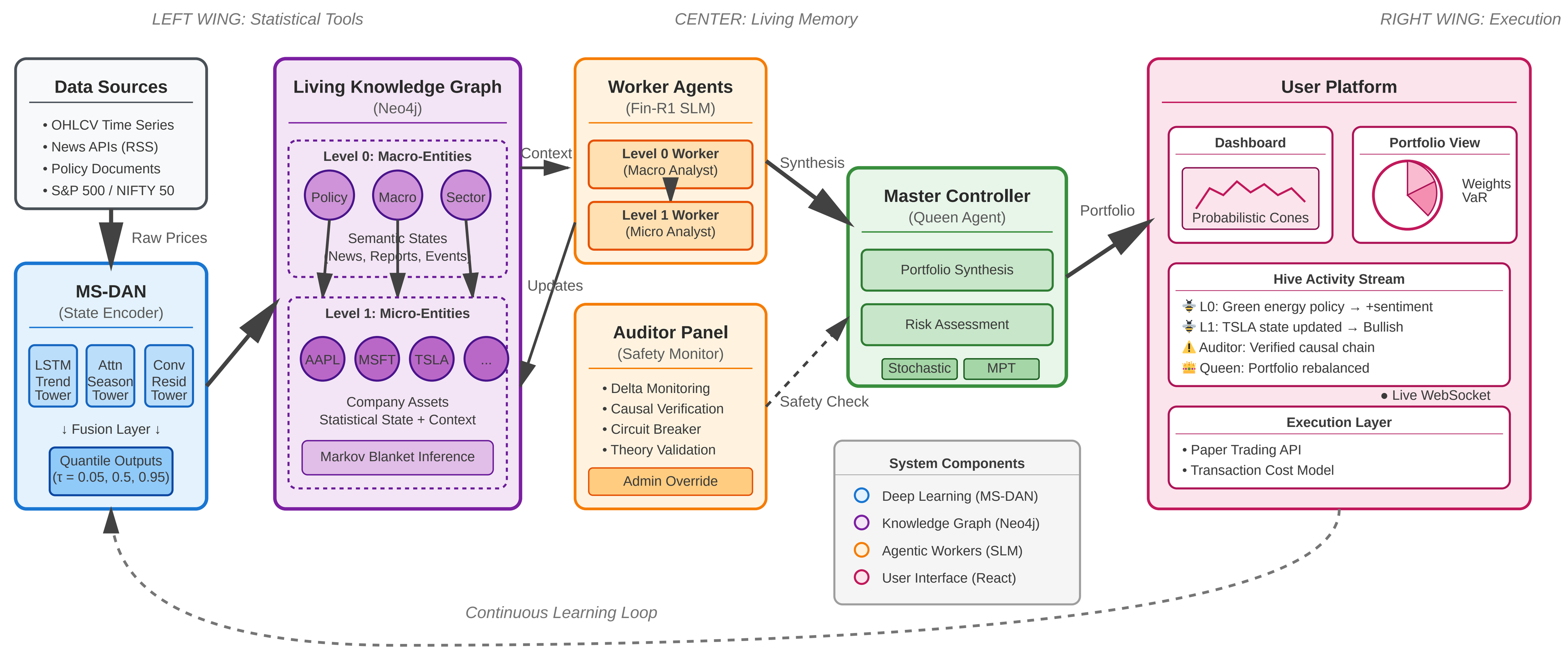
The **Multi-Scale Decompositional Attention Network (MS-DAN)** serves as the system's "State Encoder." It processes the decomposed signal through three specialized neural towers:

- **Trend Tower (Monthly)**: Uses LSTMs to capture long-term auto-regressive drift and regime persistence.
- **Seasonality Tower (Weekly)**: Uses **Sparse Multi-Head Attention** with **T5-Relative Position Embeddings (RPE)** to capture cyclical patterns invariant to absolute time.
- **Residual Tower (Daily)**: Uses 1D Convolutional Networks (Conv1D) to extract local features from high-frequency microstructure noise.
- **Fusion Mechanism**: The three embeddings are fused using a Gated Residual Network (GRN), which dynamically weighs the importance of Trend vs. Noise based on market volatility.
- **Probabilistic Output**: Trained using Pinball Loss to predict the 5th, 50th, and 95th Percentiles, creating a "Confidence Cone" rather than a single price line.

4. "The Hive" Architecture (Tripartite Model)

The system mimics a biological hive, separating "Statistics" from "Reasoning."

- **The Left Wing (The Toolset)**: Houses the MS-DAN and statistical calculators. It provides raw, deterministic state vectors (e.g., Volatility=High, Trend=Bullish).
- **The Center (The Memory)**: A Neo4j Living Knowledge Graph (LKG).
  - Structure: Hierarchical graph linking **Level 0 Nodes** (Macro entities: Inflation, Interest Rates) to **Level 1 Nodes** (Micro entities: Assets).
  - *Markov Blanket*: Inference is localized to a node's immediate neighbors to ensure computational efficiency.
- **The Right Wing (The Agents)**:
  - **Level 0 Worker**: Ingests news streams to update Macro Node states.
  - **Level 1 Worker**: Synthesizes the MS-DAN statistical vector with the Macro state to generate a trading signal.
  - **The Auditor**: A safety module that monitors "Delta" (rate of change). If predictions shift >3σ, it triggers a circuit breaker.



We currently utilize **Fin-R1**, a Large Language Model fine-tuned for financial Chain-of-Thought (CoT), to power the Worker Agents.

- **The Workflow**:
  - **Observation**: Agent receives {MS-DAN: +2% Upside, Volatility: High} AND {Graph: "Fed Rate Hike Imminent"}.
  - **Reasoning (CoT)**: "Statistical trend is positive, but macro headwinds from rate hikes usually depress growth stocks. The volatility suggests instability."
  - **Decision**: "Override statistical buy signal. Recommendation: HOLD."
- **Data Augmentation**: To improve arithmetic accuracy, we augmented the training data with "Tool-Trace" examples, teaching the model when to delegate math to Python calculators.

5. Platform Implementation

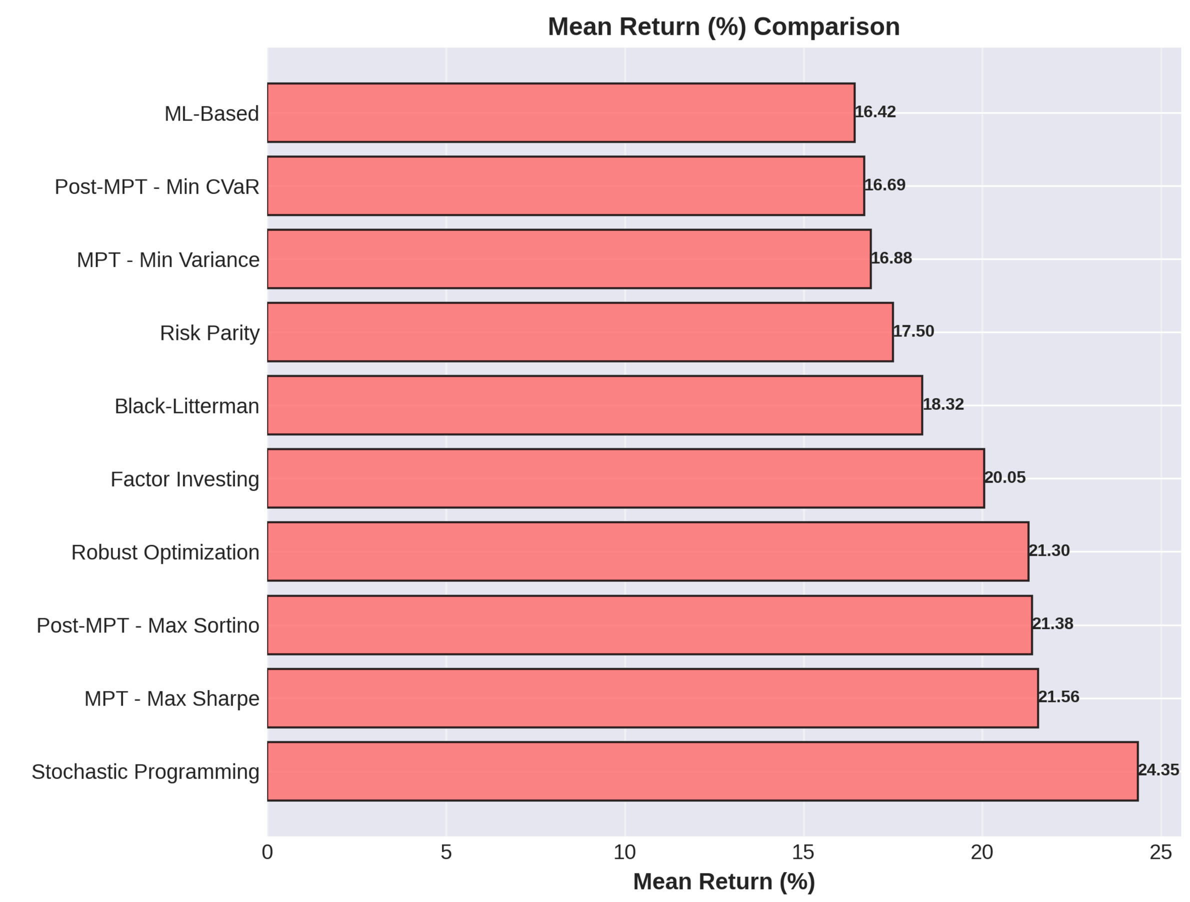
The architecture is deployed as a fully functional Fullstack Web Application.

- **Frontend**: Next.js 14 dashboard featuring real-time "Confidence Cone" visualization using Recharts/D3.js and unique UI that allows users to see not just the predicted price, but the agent's textual reasoning behind the prediction in terms of score.
- **Backend**: NodeJs core backend for traffic and supporting marketplace, with FastAPI microservice handling inference and agent orchestration.
- **Data Pipeline**: Automated ETL pipelines fetching data from Yahoo Finance and updating the PostgreSQL (Structured State Map) and Neo4j stores.

6. Performance Evaluation

We implemented **10 distinct portfolio strategies** and ran 10,000 **Monte Carlo Simulations** on historical data.

- **Stochastic Programming**: The clear winner for raw returns (**Mean Return: 24.35%**). It optimizes expected utility across multiple probable market scenarios.
- **MPT (Max Sharpe)**: The most stable baseline (**Risk-Adjusted Score: 1.22, Probability of Profit: 90.0%**).



The model was validated on the challenging COVID-19 era (high volatility).

- **Directional Accuracy**: Achieved a peak of **64.5%** on the validation set, significantly outperforming DL baseline (55%).
- **Calibration**: The model's uncertainty estimates are mathematically sound. The **95th percentile forecast** achieved a coverage probability of **~0.95**, meaning the actual price stayed within our predicted "Risk Cones" 95% of the time.
- **Impact**: This allows the system to distinguish between "High Conviction" trades (narrow cones) and "High Risk" environments (wide cones).

7. Future Work

While Fin-R1 provides strong reasoning, it lacks direct integration with external tools. Our roadmap for the next phase includes:

- **Fin-OSS Development**: Fine-tuning a bigger agentic llm. We will train it specifically on **Tool-Augmented** datasets to allow the agent to query live APIs and perform complex ratios (Sharpe/Sortino) via code execution.
- **RL Integration**: Replacing the static convex optimization solver with a **Proximal Policy Optimization** (PPO) agent. The Agent will learn to dynamically rebalance portfolio weights based on the "State Vector" provided by MS-DAN.
- **Live Paper Trading**: Connecting the system to an external API to validate the strategy in a real-time execution environment with transaction costs and slippage.